

RESUMEN INGENIERÍA DE SOFTWARE 2 - 2025

Autor: Juan Manuel Sisti.

SOFTWARE REQUIREMENTS SPECIFICATIONS

Las Especificaciones de Requisitos de Software (**SRS**) son documentos detallados que describen de manera exhaustiva los requisitos y características de un sistema de software.

Estas especificaciones actúan como un **contrato** entre los clientes, usuarios y desarrolladores, estableciendo claramente qué funcionalidades debe tener el sistema y cómo debe comportarse.

Naturaleza

El SRS es una especificación para un producto de software determinado. Es redactado por representantes del equipo de desarrollo o del cliente, o incluso por ambos.

Ambiente

El software puede contener toda la funcionalidad del proyecto o formar parte de un sistema más amplio. Es crucial describir el entorno en el cual el proyecto tendrá impacto y cómo se integrará con la funcionalidad necesaria. En el último caso, se debe detallar las interfaces entre el sistema y el software desarrollado, así como los requerimientos de funcionalidad externa.

Alcance

Define **qué incluirá y qué no incluirá** el software.

Establece las **fronteras del sistema**: funcionalidades y características que serán abordadas.

Indica **qué requerimientos se detallarán en este documento** y cuáles podrían abordarse en fases posteriores.

Características de un buen SRS (IEEE 830-1998)

Un SRS efectivo no solo se trata de enumerar una lista de requisitos, sino también de garantizar que esos requisitos sean claros, coherentes y comprensibles tanto para el equipo de desarrollo como para el resto de los stakeholders involucrados en el proyecto.

Un SRS **útil** debe cumplir con los siguientes criterios de calidad:

Característica	Descripción
Correcto	Es correcto sólo si cada requisito declarado está presente en el software.
No ambiguo	Es inequívoco sólo si cada requisito declarado tiene una única interpretación.
Completo	Está completo solo si cubre todos los aspectos necesarios para que el sistema funcione correctamente y cumpla con los objetivos establecidos.
Consistente	La consistencia se refiere a la consistencia interior. No hay contradicciones internas ni con documentos superiores.
Priorizado	Se prioriza por la importancia de sus requerimientos particulares.
Comprobable	Es comprobable sólo si cada requisito declarado es verificable. Un requisito es verificable si existe un proceso mediante el cual una persona o una máquina puede verificar que el producto de software cumple con el requisito.
Modificable	Es modificable sólo si su estructura y estilo permiten realizar cambios a los requerimientos de manera sencilla, completa y coherente.
Trazable	Se refiere a su capacidad de ser rastreado hacia adelante y hacia atrás en todo el proceso de desarrollo del software. Es posible rastrear el origen y evolución de cada requerimiento.

Consideraciones importantes para elaborar el SRS

Evolución: el documento debe evolucionar de manera paralela al desarrollo del software. Debe registrar **quién hizo qué cambios** y asegurar la aceptación de las modificaciones por parte de las partes involucradas.

Prototipos: pueden utilizarse para definir y refinar los requerimientos de manera más precisa, permitiendo a los stakeholders visualizar y experimentar cómo se verá y funcionará la implementación.

Diseño incorporado: El SRS puede incluir atributos o funciones externas que el sistema debe tener para interactuar con otros subsistemas o componentes. Esto puede involucrar detalles de diseño

Requisitos complementarios: a veces, ciertos detalles se documentan en **anexos** como casos de uso, planes de proyecto, o plan de aseguramiento de calidad.

GESTIÓN DE PROYECTOS

Es el proceso de planificar, organizar, dirigir y controlar recursos y actividades para alcanzar los objetivos de un proyecto, respetando tiempo, costo, calidad y alcance.

Proyecto

- Esfuerzo temporal con inicio y fin definidos.
- Crea un producto, servicio o resultado único.
- Se desarrolla gradualmente y se divide en tareas simples.

Las 4 P de la Gestión

1. **Personal (RRHH):** El Factor humano es fundamental en la Gestión de Proyectos de software ya que son quienes realizan el esfuerzo. Se gestiona su participación según capacidades y roles.
2. **Producto:** Intangible. Planificar el proyecto con claridad es crucial para establecer objetivos y alcances precisos del producto.
3. **Proceso:** Brinda la estructura desde la cual se puede establecer un plan detallado para el desarrollo del software.
4. **Proyecto:** Su éxito depende de planificación, control, asignación de recursos y mitigación de riesgos.

Mala gestión

- Incumplimiento de plazos.
- Incremento de los costos.
- Entrega de productos de mala calidad.

Elementos clave

- Métricas
 - Estimaciones
 - Planificación (temporal y organizativa)
 - Análisis de riesgos
 - Seguimiento y control
-

MÉTRICAS

Permiten medir, evaluar y mejorar procesos, productos y proyectos de software.

Son herramientas para que profesionales e investigadores tomen mejores decisiones. Sirven para medir procesos, proyectos y productos de software, entre otros, y garantizan la calidad.

Definiciones:

- **Medida:** Valor cuantitativo de un atributo como tiempo, recursos, líneas de código, etc.
- **Medición:** Es el proceso de determinar una medida a través de cálculos y análisis.
- **Métrica:** Son herramientas que te permiten obtener información más significativa a partir de las medidas.
- **Indicador:** Elemento de información, a partir de la combinación de métricas para establecer la pauta de cuál es la medida correcta.

Tipos de Métricas

1. **Del Proyecto:** Se centran en medir el progreso y el rendimiento inmediato de un proyecto específico

Evalúa progreso, uso de tiempo, tareas, RRHH, errores, riesgos y áreas problemáticas.

2. **Del Proceso:** Se utilizan para evaluar y mejorar la eficiencia, calidad y efectividad de los procedimientos y flujos de trabajo en una organización.

Evalúa eficiencia y calidad a largo plazo. Busca mejora continua.

Conocimiento, Retroalimentación, Evaluaciones, Claridad, Ampliación.

3. **Del Producto:**

Estáticas:

Una métrica estática del producto se refiere a una medida cuantitativa que se obtiene a partir del análisis de las representaciones estáticas del software, como el código fuente, la arquitectura, los diagramas y la documentación. Estas métricas se utilizan para evaluar aspectos relacionados con la estructura, el diseño y la complejidad del software. Sobre código o diseño (Fan-in/out, complejidad ciclomática, LDC, Profundidad de anidamientos, etc).

Algunas métricas estáticas:

Fan-in/Fan-out Fan-in:

Es una medida del número de funciones que llaman a una función X. Fan-out es el número de funciones que son llamadas por una función X. Cuando una función tiene un alto Fan-in, significa que está fuertemente acoplada al resto del diseño. Un alto Fan-out sugiere que la función podría tener una alta complejidad por la coordinación en cada llamada.

Longitud del código:

Medida del tamaño del programa. Entre más extenso sea el código, es más complejo y susceptible a errores.

Complejidad ciclomática:

Medida de la complejidad del control de un programa. Está relacionada con la comprensión del programa. Longitud de los identificadores Medida de la longitud promedio de los diferentes identificadores en un programa. Entre más grande sea su longitud, serán más fáciles de distinguir y por lo tanto el programa será más comprensible.

Longitud de los identificadores:

Medida de la longitud promedio de los diferentes identificadores en un programa. Entre más grande sea su longitud, serán más fáciles de distinguir y por lo tanto el programa será más comprensible.

Profundidad del anidamiento de las condiciones:

Medida de profundidad de anidamiento de las instrucciones condicionales en un programa. Un gran anidamiento dificulta la comprensión y aumenta la probabilidad de errores.

Índice de Fog:

Medida de la longitud promedio de las palabras y las frases en los documentos. Cuanto más grande sea el índice Fog, el documento será más difícil de comprender.

Dinámicas:

Las métricas dinámicas del producto se relacionan con las características de calidad del software y son indicadores de rendimiento y comportamiento que se generan y evalúan en tiempo real. Son esenciales para comprender el rendimiento en tiempo real (Tiempo de respuesta, Fiabilidad, Usabilidad).

Atributos

Los atributos que miden la calidad del producto se pueden diferenciar entre atributos de calidad externos (mediante métricas dinámicas) y atributos internos (mediante métricas estáticas). Se busca que los atributos internos contribuyan a cumplir los atributos externos. Los atributos de calidad externos están relacionados con los requerimientos no funcionales del producto. Los atributos internos están dentro de las métricas estáticas y son características o propiedades que no son directamente observables por los usuarios finales, sino que están relacionadas con la estructura, el diseño, el código o los componentes internos del producto.

ESTIMACIONES

Son técnicas que permiten asignar valores aproximados a tiempo, costos o recursos cuando no se dispone de datos precisos. Se utilizan para proporcionar una idea general y rápida de lo que se espera.

No son una ciencia exacta. Pueden ser precisas o no, y factores imprevistos pueden alterarlas. Las estimaciones también requieren experiencia para realizarlas de manera efectiva.

A diferencia de las métricas, que son medidas seguras y precisas, las estimaciones son valores aproximados y más subjetivos.

Tipos:

- **Recursos:**

Recursos Humanos (Personal):

Cantidad necesarias para llevar a cabo el proyecto, Habilidades requeridas para cada rol en el proyecto, Duración de cada miembro para completar sus tareas., Ubicación, dónde se encuentra físicamente el equipo de trabajo

Entorno:

Herramientas de Software: Identificar y estimar el uso de herramientas de software necesarias para el desarrollo y la gestión del proyecto.

Hardware: prever el hardware requerido, como servidores, dispositivos móviles, computadoras, etc.

Recursos de Red: Evaluar la infraestructura de red necesaria para la colaboración y el acceso a recursos compartidos.

Software Reutilizable:

Componentes COTS: Evaluar adquisición de componentes de software que proveen una funcionalidad específica para ser adquiridos e integrados dentro del sistemas de software.

Componentes Nuevos: Identificar cualquier nuevo componente de software que necesite ser desarrollado específicamente para el proyecto.

Componentes de Experiencia Completa: Incluir componentes que ya se hayan utilizado o desarrollado en proyectos anteriores y que se puedan aprovechar nuevamente.

Componentes de Experiencia Parcial: Considerar componentes que se hayan desarrollado parcialmente y requieran ajustes y pruebas adicionales.

- **Tiempo:**

Estima la duración de tareas utilizando la técnica **PERPSDG** (Personas, Entorno, Requerimientos, Producto, Especificación, Diseño, Gestión).

Al utilizar esta técnica se pueden realizar estimaciones más sólidas y desarrollar un cronograma que sea realista y adaptable a las circunstancias cambiantes del proyecto.

- **Costos:**

Costo del esfuerzo: relacionados con los recursos humanos, específicamente con los salarios y remuneraciones de los miembros del equipo de desarrollo.

Hardware y Software: costos asociados con la infraestructura tecnológica. Esto abarca el hardware necesario, como servidores, dispositivos móviles y equipos de cómputo.

Viajes: involucran los gastos relacionados con la movilidad del equipo, como los viajes necesarios para reuniones, colaboraciones o para estar presente en diferentes ubicaciones.

Relación Precio-Costo

El precio de un producto o servicio puede diferir del costo real. El precio se ve influido por el mercado, competencia, tipo de contrato, riesgos asumidos, salud financiera de la organización y otros factores económicos.

Técnicas

- **Juicio experto:** Se consulta a varios expertos en el campo relevante para obtener sus opiniones y estimaciones. Los expertos estiman, comparan y discuten entre sí para llegar a una solución conjunta.
- **Planning Poker:** Técnica colaborativa para metodologías ágiles. Los integrantes del equipo asignan puntos a tareas mediante cartas con valores (basados en serie de Fibonacci). Se discuten discrepancias hasta llegar a consenso.

- **COCOMO II (Constructive Cost Model):**

Se basa en datos históricos de un gran número de proyectos pasados y utiliza fórmulas para relacionar el tamaño del sistema y los factores del producto, proyecto y equipo con el esfuerzo para realiza el sistema.

- Modelos:

- **Composición de aplicación:** Basado en componentes reutilizables. Estima el esfuerzo en función de puntos de aplicación
- **Diseño temprano:** Se usa al inicio del proyecto, cuando hay poca información, útil en las primeras etapas del proyecto, antes de tener un diseño arquitectónico detallado.
- **Reutilización:** Este modelo se basa en el número de líneas de código de reutilización o generadas automáticamente.
- **Post-arquitectura:** Basado en el número de líneas de código fuente, se emplea cuando existe un diseño arquitectónico inicial.

Se hace en base a tres componentes: Líneas nuevas de código. Líneas equivalentes de código fuente (ESLOC) basadas en reutilización. Líneas modificadas debido a cambios en los requerimientos.

PLANIFICACIÓN TEMPORAL

Es un aspecto esencial en la gestión de proyectos. Busca establecer una secuencia lógica de tareas y asignar los recursos adecuados a cada una, con el objetivo de cumplir con los plazos y metas del proyecto de manera eficiente.

Una precisa planificación temporal resulta crucial para evitar la insatisfacción de los clientes, la generación de costos adicionales y la disminución del impacto en el mercado.

Calendarización

Implica la distribución del esfuerzo estimado a lo largo del período planificado para un proyecto, asignando dicho esfuerzo a tareas específicas del desarrollo de un software. La designación de la fecha final de un proyecto puede realizarse en dos escenarios diferentes:

Fecha final establecida por el cliente: En este caso, el equipo está obligado a distribuir el esfuerzo dentro del plazo establecido por el cliente. Los recursos deben ajustarse para cumplir con esa fecha.

Fecha final fijada por los desarrolladores: En este escenario, el esfuerzo se distribuye de manera que se logre una utilización óptima de los recursos. Se determina una fecha de finalización después de un análisis exhaustivo. Desafortunadamente, la primera opción es la más comúnmente encontrada.

Acciones ante retrasos

1. Revisar impacto sobre la fecha de entrega: algunas tareas pueden ser críticas y tener un impacto significativo
2. Reasignar recursos: se puede considerar la posibilidad de reasignar recursos para acelerar el trabajo, no siempre resulta en un aumento directo de la productividad.
3. Reordenar tareas: en algunos casos es posible reorganizar la secuencia de las tareas para minimizar el impacto de los retrasos, tarea cuidadosa para evitar problemas adicionales.
4. Ajustar entregas: en algunos casos se podría considerar la posibilidad de ajustar la fecha de entrega o la entrega parcial de resultados. Esto podría implicar negociaciones con los stakeholders del proyecto para establecer expectativas realistas

Métodos

- **Gantt:** Gráfico de barras (en desuso).
- **PERT:** Método probabilístico (tiempos optimistas/pesimistas).

Este método propone la creación de una red de tareas, considerando tiempos más probables, optimistas y pesimistas para completar cada tarea. Utiliza conceptos como fechas tempranas, fechas tardías y camino crítico para la gestión de proyectos.

- **CPM:** Método determinístico (tiempos definidos).

A diferencia de PERT, se enfoca en proyectos donde las estimaciones de tiempo son más concretas. Además de la red, se trabaja con tiempos de inicio temprano y tiempos de inicio tardío para cada tarea

- **Camino Crítico (PERT-CPM):** Fusiona ambos, identifica tareas sin margen de demora.

Este enfoque integrado combina las fortalezas de ambas metodologías para brindar una herramienta más completa y versátil en la planificación y gestión de proyectos. El Camino Crítico se compone de las tareas que no tienen margen de tiempo adicional para retrasos, es decir, cualquier demora en una tarea crítica retrasará la finalización del proyecto en su totalidad. Por lo tanto, identificar el Camino Crítico es esencial para la gestión efectiva del tiempo y los recursos en un proyecto.

PLANIFICACIÓN ORGANIZATIVA

Se refiere a la gestión del recurso humano en un proyecto. El personal es fundamental y su manejo adecuado es un factor crítico para el éxito del proyecto. El equipo de una organización de software es un activo valioso y representa el capital intelectual. Una mala gestión del personal puede ser uno de los factores principales para el fracaso de los proyectos.

Participantes

Gerentes ejecutivos: Definen los temas empresariales. Gerentes de proyecto: Planifican, motivan, organizan y controlan a los profesionales. Profesionales: Aportan habilidades técnicas. Clientes: Especifican los requerimientos. Usuarios finales: Interactúan con el software.

Líderes

Desempeña un papel fundamental en la gestión y dirección de un grupo de individuos que trabajan juntos hacia un objetivo común. Tiene la responsabilidad de guiar, coordinar, motivar y supervisar a los miembros del equipo para asegurarse de que trabajen de manera efectiva y alcancen los resultados deseados.

- Modelo **MOI** de Liderazgo:

Motivación al personal: Habilidad para incentivar al personal técnico a dar su máximo rendimiento.

Organización del equipo: Habilidad para diseñar procesos que conduzcan al producto final.

Innovación: Capacidad para fomentar la creatividad y la generación de ideas del personal.

Estructura de un equipo de software

No existe una única estructura ideal para todos los equipos. La forma en que se organiza un equipo de desarrollo depende de varios factores contextuales.

Factores que influyen en la estructura del equipo:

1. **Dificultad del problema:** Cuanto más complejo es, más especializada debe ser la estructura.
2. **Tamaño del programa:** Proyectos más grandes requieren equipos más amplios y roles más definidos.
3. **Duración del equipo:** Si el equipo trabajará junto por mucho tiempo, se puede invertir más en cohesión y roles estables.
4. **Modularidad del problema:** Si el sistema puede dividirse en módulos, es más fácil distribuir tareas y formar subequipos.
5. **Calidad y confiabilidad requeridas:** Proyectos críticos requieren controles más estrictos y perfiles experimentados.
6. **Rigidez de la fecha de entrega:** Plazos estrictos demandan mejor coordinación, metodologías ágiles o equipos más grandes.
7. **Grado de sociabilidad:** Algunos proyectos requieren alto nivel de comunicación y colaboración, lo que influye en la estructura y dinámica del equipo.

Comunicación en Equipos de Software

La comunicación efectiva es un factor crítico en el rendimiento del equipo de desarrollo. Su calidad puede verse afectada por:

- El estatus y rol de los miembros.

- El tamaño y composición del grupo (género, personalidad).
- Los canales y herramientas de comunicación disponibles.

Para maximizar la productividad, es esencial ofrecer un entorno de trabajo con **recursos adecuados y espacios que faciliten la interacción**, promoviendo una colaboración fluida y eficiente entre los integrantes del equipo.

RIESGOS

Un **riesgo** es un evento no deseado con consecuencias negativas. Los **gerentes de proyectos** deben anticipar su ocurrencia y elaborar planes para **prevenir o minimizar su impacto**.

Componentes

Incertidumbre (probabilidad): Posibilidad de que ocurra un riesgo (nunca 100%).

Pérdida (impacto): Consecuencias negativas posibles (leve a catastrófico).

Preocupación por el futuro: Riesgos que podrían llevar al fracaso del proyecto.

Impacto de los cambios: Cambios en requisitos o tecnologías afectan el proyecto.

Consideraciones en decisiones: Métodos, herramientas, RRHH, calidad.

Factores que Aumentan el Riesgo

- Falta de reuniones entre gerentes y clientes.
- Usuarios no comprometidos.

- Requisitos mal entendidos o no definidos claramente.
- Expectativas poco realistas.
- Ámbito inestable.
- Falta de habilidades en el equipo.
- Requisitos o tecnologías nuevas no dominadas.
- Tamaño inadecuado del equipo.
- Falta de consenso entre stakeholders.

Tipos de Riesgos

	GENÉRICOS	ESPECÍFICOS
Características	Estos riesgos son independientes del dominio en el que se trabaje y pueden aplicarse a cualquier proyecto.	Estos riesgos están relacionados con el dominio específico de desarrollo de software.
Conocidos	Son riesgos que se han experimentado en otros proyectos similares.	Son riesgos que los expertos en el dominio saben que podrían ocurrir.
Predecibles	Son riesgos que son fáciles de anticipar, como cambios en los requisitos.	Son riesgos que se pueden imaginar fácilmente en el contexto del dominio.
Impredecibles	Son riesgos difíciles de prever y pueden surgir en cualquier proyecto.	Son riesgos que podrían surgir, pero son muy difíciles de anticipar.

Administración de Riesgos

Proceso para **identificar, evaluar, planificar y controlar** riesgos. Su objetivo es minimizar probabilidad e impacto de eventos adversos.

Estrategias

- **Proactivas:** Enfoque en anticipar y prevenir problemas. Se reconoce que los riesgos pueden surgir y se planifican estrategias de tratamiento para evitar que los riesgos tengan un impacto crítico.

- **Reactivas:** Enfoque en reaccionar ante un problema una vez que ocurre, manejando la crisis en el momento. No hay prevención, solo se toma acción cuando el problema se presenta.

Proceso de Gestión de Riesgos

El proceso de gestión de riesgos consta de 4 pasos: identificación, análisis, planeación y supervisión. Este proceso es iterativo, ya que después de cada supervisión, es posible que sea necesario reevaluar los riesgos.

1. Identificación

- Listado de riesgos potenciales mediante brainstorming.
- Se usan listas de chequeo y se clasifican (proyecto, producto, negocio).

2. Análisis

- Evaluar **probabilidad e impacto** de cada riesgo.
- Clasificación:

■ Probabilidad:

1. <10%: Bastante improbable
2. 10-25%: Improbable
3. 25-50%: Moderado
4. 50-75%: Probable
5. 75%: Bastante probable

- **Impacto:**

1. Insignificante
2. Tolerable
3. Serio
4. Catastrófico

- Se priorizan riesgos en una tabla.
- **Línea de corte:** Define cuáles serán tratados primero.
- Según Boehm, atender los 10 riesgos más importantes.

3. Planeación

- En esta etapa, se abordan los riesgos que están por encima de la línea de corte (los más prioritarios) y se determina la estrategia a seguir:
 - **Evitar:** Rediseñar para que no ocurra.
 - **Minimizar:** Reducir su probabilidad.
 - **Contingencia:** Plan para mitigar efectos si ocurre.
- Evaluar **costos** de las estrategias. Empresas chicas tienden a ser reactivas por presupuesto. Se aplica si el valor de **influencia** justifica la inversión.

4. Supervisión

- Reevaluar riesgos en cada etapa del proyecto:
 - ¿Cambió la probabilidad?
 - ¿Funcionan las estrategias?

- ¿Surgieron nuevos riesgos?
- ¿Se cumplieron los planes de mitigación?
- Es un proceso **iterativo**, con retroalimentación constante. Tras esta etapa, algunos riesgos vuelven a la etapa de análisis ya que es un proceso iterativo con constantes cambios a partir de la evolución del proyecto.

DISEÑO DE SOFTWARE

Proceso mediante el cual se define cómo se organizarán los **componentes, módulos, funciones** e interacciones del sistema, partiendo de los **requisitos del sistema** y las **necesidades del usuario**.

Un diseño deficiente puede llevar a un sistema inestable y difícil de mantener.

Áreas del Diseño

- **Diseño de datos:** Define estructuras, objetos y relaciones para almacenar y gestionar los datos en el sistema.
- **Diseño arquitectónico:** Organiza los componentes estructurales, selecciona estilos arquitectónicos y patrones de diseño y otras decisiones clave para la estructura general del sistema.
- **Diseño a nivel de componentes:** Detalla los componentes de software y su comportamiento. Se basa en modelos de clases, flujos y comportamientos para definir cómo funcionarán estos componentes.
- **Diseño de interfaz:** Establece la comunicación interna y externa del sistema, incluyendo la interacción con el usuario.

Cumplimiento y Calidad

Un buen diseño debe tener los siguientes puntos:

- **Cumplimiento:** Implementar requisitos explícitos e implícitos para satisfacer las necesidades del cliente.
- **Legible:** Ser legible y entendible por todos los involucrados.
- **Completo:** Debe contemplar todos los componentes, interacciones y funcionalidades necesarias.

Criterios para Evaluar el Diseño

- Arquitectura basada en patrones reconocibles.
- Modularidad con componentes cohesivos e independientes.
- Múltiples representaciones para abordar diferentes aspectos del sistema.
- Estructuras de datos adecuadas.
- Interfaces simples y bien diseñadas.
- Proceso iterativo y controlado por los requisitos.
- Trazabilidad a los requerimientos.
- Equilibrio entre diseño de datos y funciones.
- Facilidad de uso e independencia funcional.
- Bajo acoplamiento.
- Representaciones comprensibles.
- Compatible con metodologías ágiles.

Evolución y Conceptos del Diseño

- Traduce el modelo de requisitos en representaciones técnicas.
- Usa notaciones específicas para componentes e interfaces.
- Aplica refinamiento heurístico para mejorar el diseño gradualmente.
- Evalúa calidad según lineamientos definidos.

Conceptos Fundamentales

- **Abstracción:** Representa elementos sin entrar en detalles. Tipos:
 - Procedimental: Agrupa instrucciones bajo un nombre.
 - De datos: Agrupa datos con un propósito.
- **Arquitectura del software:** Estructura global del sistema.
- **Patrones:** Soluciones reutilizables y efectivas para problemas comunes.
- **Modularidad:** Divide el sistema en módulos equilibrando cantidad y complejidad.
- **Ocultamiento de información:** Limita el acceso a los detalles internos del módulo.
- **Independencia funcional:** Módulos con baja dependencia y responsabilidades claras.
 - Evaluada por **cohesión** (alta deseable) y **acoplamiento** (bajo deseable).

Cohesión

Refleja qué tan centrado está un módulo en una sola tarea.

- Tipos: coincidental, lógica, temporal, procedimental, comunicacional, funcional.

Acoplamiento

Grado de dependencia entre módulos. Ideal: bajo.

- Tipos:
 - **Bajo**: de datos, de marca.
 - **Moderado**: de control.
 - **Alto**: común, externo, de contenido.

Refinamiento

Proceso gradual de llevar el diseño desde una visión general hasta un nivel de detalle procedimental. Conforme se avanza, se trabaja en cada iteración con un mayor nivel de detalle, lo que permite adentrarse en el funcionamiento concreto de la funcionalidad en cuestión. Se complementa con la abstracción.

Refabricación (Refactoring)

Técnica de reorganización que tiene como objetivo simplificar el diseño de un componente sin alterar su función ni su comportamiento. Esta técnica se emplea especialmente en metodologías ágiles y a lo largo de las distintas iteraciones del proceso de desarrollo. Su propósito principal es permitir que los módulos que lo requieran puedan alcanzar niveles más altos de cohesión y acoplamiento, a través de modificaciones internas. Durante el proceso de refabricación, se analizan elementos como redundancias, componentes superfluos, algoritmos innecesarios y estructuras inapropiadas.

DISEÑO ARQUITECTÓNICO

El diseño arquitectónico define las relaciones entre los elementos estructurales para cumplir con los requisitos del sistema. Una vez que los elementos del sistema han sido identificados, el sistema arquitectónico se encarga de establecer las conexiones entre ellos para satisfacer los requisitos.

REQUISITOS NO FUNCIONALES

La arquitectura tiene un impacto directo en los requisitos no funcionales, siendo los más críticos **el rendimiento, la seguridad, la protección, la disponibilidad y la mantenibilidad**.

Rendimiento: Es necesario agrupar las operaciones críticas en un reducido número de subsistemas (componentes de granularidad gruesa y baja comunicación).

Seguridad: Se debe emplear una arquitectura en capas para proteger los recursos críticos en las capas internas y menos expuestas.

Protección: La arquitectura debe ser diseñada de modo que las operaciones relacionadas con la protección se encuentren en un único subsistema con el fin de reducir los costos y los problemas de validación de la protección.

Disponibilidad: La arquitectura debe ser diseñada con componentes redundantes, lo que posibilita el reemplazo sin interrumpir el funcionamiento del sistema.

Mantenibilidad: La arquitectura del sistema debe contemplar componentes autocontenidos de granularidad fina que puedan ser modificados con facilidad.

PATRONES DE DISEÑO

Los patrones arquitectónicos son estilos organizacionales utilizados en diversas arquitecturas de software. Algunos de los más comunes incluyen: repositorio, cliente-servidor, arquitectura en capas o combinaciones entre ellos, entre otros.

PATRÓN DE REPOSITORIO

El patrón de repositorio se basa en la organización de un sistema en torno a una base de datos central compartida, conocida como repositorio. En este enfoque, todos los subsistemas acceden a los datos a través de este repositorio central.

Es comúnmente empleado en sistemas que manejan grandes volúmenes de datos que necesitan ser compartidos y accedidos por varios subsistemas. Es especialmente

efectivo en entornos donde se requiere una fuente única y centralizada de verdad para los datos.

PATRÓN DE CLIENTE-SERVIDOR

El patrón cliente-servidor es un modelo arquitectónico en el que el sistema se organiza como un conjunto de servicios y servidores asociados, junto con clientes (interfaces) que utilizan estos servicios a través de una red

Componentes Servidores de Red: Proveedores de servicios que ofrecen funcionalidades específicas. Clientes: Consumidores de servicios que solicitan y utilizan las funciones proporcionadas por los servidores. Red: Infraestructura que permite a los clientes acceder a los servicios a través de la comunicación en la red.

Es poderoso para dividir responsabilidades y permitir escalabilidad y reutilización, pero también introduce desafíos relacionados con la complejidad, la dependencia de la red y la seguridad.

Arquitectura de los Sistemas Distribuidos.

Es una infraestructura donde componentes de software y hardware están distribuidos en diferentes computadoras conectadas en red, trabajando en conjunto para cumplir un objetivo común. El procesamiento de información se reparte entre distintos nodos, lo que permite:

Tipos comunes:

- **Cliente-Servidor**
- **Componentes Distribuidos**

Ventajas

- **Recursos compartidos:** uso eficiente de procesamiento, memoria, almacenamiento.
- **Apertura:** diseño con protocolos estándar, facilita integración.

- **Concurrencia:** múltiples procesos en paralelo.
- **Escalabilidad:** vertical u horizontal.
- **Tolerancia a fallos:** continúa funcionando si un nodo falla.

Desventajas

- **Complejidad:** requiere gestionar sincronización y comunicación.
- **Seguridad:** múltiples puntos de acceso generan vulnerabilidades.
- **Manejabilidad:** diversidad de hardware/sistemas dificulta administración.
- **Impredecibilidad:** comportamiento variable según carga/red.

Arquitectura Multiprocesador

Conforma un sistema con múltiples procesadores o núcleos en un mismo equipo. Permite ejecutar procesos en paralelo. La **asignación de procesos** puede ser:

- **Predefinida** por criterios.
- **Dinámica**, manejada por un **dispatcher** que optimiza recursos.

Usada en sistemas de **tiempo real** y también en **PCs y smartphones**, mejorando rendimiento.

Arquitectura Cliente-Servidor

En la arquitectura Cliente-Servidor, se diseña el sistema como un conjunto de servicios proporcionados por servidores y un conjunto de clientes que acceden a estos servicios a través de una red de comunicación.

Funcionan de forma concurrente y en máquinas separadas. Los servidores pueden ofrecer múltiples servicios y atender varios clientes a la vez. Existen:

- **Clientes ligeros:** solo presentan la información. El procesamiento y la gestión de datos más complejos o de aplicación son realizados principalmente por el servidor
- **Clientes pesados:** Se encarga de parte del procesamiento de la aplicación y la lógica de presentación, mientras que el servidor se concentra en la gestión de datos.

Niveles de arquitectura:

- **Dos niveles:** cliente-servidor.
- **Multinivel:** presentación, lógica de negocio, base de datos.

Los **servidores** pueden comunicarse entre sí para compartir datos y coordinar operaciones.

Arquitectura de Componentes Distribuidos

Diseña el sistema como un conjunto de **componentes u objetos distribuidos**, donde cada componente puede ser cliente y servidor a la vez. En esta arquitectura, los componentes pueden residir en diferentes máquinas y equipos de cómputo, conectados a través de una red.

- Utiliza **middleware** como intermediario para gestionar peticiones.
- Los servicios son ofrecidos mediante **interfaces específicas**.

Computación Distribuida Inter-Organizacional

Permite distribuir tareas entre **servidores de distintas organizaciones**, compartiendo recursos y cooperando computacionalmente.

Peer-to-Peer (P2P)

En los sistemas Peer-to-Peer, la computación se puede realizar en cualquier nodo de la red, y no hay un nodo centralizado que controla todo el proceso.

En esta arquitectura:

- Todos los nodos pueden ser clientes y servidores.
- El procesamiento y almacenamiento es distribuido.

Tipos:

- **Descentralizado:** sin nodo central, todos los nodos actúan como iguales.
- **Semi-centralizado:** hay un nodo de coordinación, pero no controla toda la red.

Ejemplo: intercambio de archivos por torrents.

Arquitectura Orientada a Servicios (SOA)

Basada en representar funciones como **servicios reutilizables e independientes** que pueden ser usados por múltiples aplicaciones.

- **Proveedor de servicio** registra su interfaz en un **registro de servicios**.
- **Solicitante** enlaza ese servicio a su aplicación, lo invoca y procesa el resultado.

Promueve la **reutilización, interoperabilidad y flexibilidad** en la construcción de sistemas.

IMPLEMENTACIÓN

Una vez establecido el diseño, la siguiente etapa es la codificación del mismo. Esto implica la escritura de los programas que implementarán dicho diseño

Pautas generales de la codificación

Para **mantener la calidad del diseño** durante la codificación, se recomienda seguir estas prácticas:

- **Localización de entrada y salida:** Es preferible ubicar la entrada/salida en componentes separados del resto del código, ya que suelen ser más difíciles de probar.
- **Utilización de pseudocódigo:** Puede servir de puente entre el diseño abstracto y el lenguaje de programación elegido.
- **Revisión y reescritura:** Es aconsejable generar un primer borrador del código, **revisarlo** y **reescribirlo** las veces que sea necesario. No se trata solo de corregir pequeños errores, sino de reestructurar partes para asegurar claridad, eficacia y calidad.
- **Reutilización productiva:** Consiste en crear nuevos componentes con la intención de que puedan ser **reutilizados** por otras aplicaciones, módulos o sistemas.
- **Reutilización consumidora:** Se refiere al uso de **componentes ya existentes**, desarrollados para otros proyectos. En vez de crear algo desde cero, se aprovechan **repositorios o bibliotecas** que satisfacen la necesidad actual.

Documentación en la codificación

La documentación cumple un rol esencial durante la implementación, ya que proporciona **descripciones escritas** que explican la funcionalidad y el funcionamiento del programa.

Su objetivo principal es **facilitar la comprensión**, futuras modificaciones, correcciones y ampliaciones del código.

Tipos de documentación:

Documentación interna

- Destinada a **programadores**.

- De carácter **técnico y conciso**.
- Describe **algoritmos, estructuras de control, flujos de ejecución**, entre otros aspectos internos.
- Es clave para el **mantenimiento, depuración** y trabajo en equipo.

Documentación externa

- Destinada a **diseñadores, evaluadores u otros perfiles** que tal vez no accedan al código fuente.
 - Se enfoca en **interfaces y funcionalidades** del sistema.
 - Permite a los usuarios comprender **cómo interactuar con el programa y qué resultados esperar**, sin conocer detalles técnicos.
-

DISEÑO DE INTERFAZ DE USUARIO (UI)

La **interfaz de usuario (UI)** define cómo las personas interactúan con sistemas informáticos, aplicaciones, dispositivos móviles y sitios web. El enfoque central está en la **experiencia del usuario** y la comunicación entre el humano y la tecnología.

Es una actividad **multidisciplinaria**, que involucra diseño gráfico, industrial, web, de software, y ergonomía. Se aplica en múltiples entornos: computadoras, vehículos, aviones, etc.

Objetivo

Crear una **interacción atractiva y centrada en el usuario**, facilitando el aprendizaje sobre el uso del sistema. El diseño gráfico e industrial se combinan para optimizar esta interacción, usando símbolos, pictogramas y elementos visuales, sin afectar la eficiencia técnica.

Debe **adaptarse a los usuarios**, no al revés. La interfaz debe facilitar el acceso a información compleja sin sacrificar la comprensión, evitando errores por una mala presentación o complejidad.

Principios para el diseño UI

1. **Análisis de actividades del usuario**
2. **Diseño de prototipos en papel**
3. **Evaluación con usuarios finales**
4. **Diseño dinámico del prototipo**
5. **Nueva evaluación con usuarios**
6. **Implementación de Prototipo Funcional**

Proceso del Diseño UI

1. **Análisis y Modelado:** Se definen elementos y acciones de la interfaz según el análisis previo.
 2. **Diseño de la Interfaz:** Se representan los eventos o acciones esperadas del usuario.
 3. **Construcción de la Interfaz:** Se crean los estados que vivirá la UI durante la interacción.
 4. **Validación:** Se verifica cómo el usuario interpreta el estado del sistema. Se repite hasta lograr una UI efectiva.
-

Diseño de Experiencia de Usuario (UX)

Es el conjunto de métodos para **satisfacer las necesidades del cliente** mediante **interacciones significativas y satisfactorias**. Abarca desde la primera impresión hasta el uso continuo del sistema.

Incluye:

- **Usabilidad**
- **Accesibilidad**
- **Diseño visual**
- **Arquitectura de la información**
- **Empatía con el usuario**

Busca ofrecer un entorno **intuitivo y comprensible**, evitando transiciones bruscas. Fomenta interacciones fluidas y agradables que se alineen con las expectativas y necesidades de los usuarios.

Tipos de Diseño UX

- **Diseño de Interacción:** Cómo interactúa el usuario con el sistema.
- **Diseño Visual:** Apariencia estética, uso de color, espacio, forma.
- **Investigación del Usuario:** Conocer sus necesidades, deseos, comportamientos.
- **Arquitectura de la Información:** Organizar el contenido eficientemente (ejemplo: mapa de subte).

Etapas del Diseño UX

1. **Investigación:** Recolectar información sobre usuarios, productos y competencia.

2. **Organización:** Convertir datos en una base estructurada (mapas de empatía, experiencia).
3. **Diseño:** Crear prototipos, flujos e interfaces alineados con los objetivos del proyecto.
4. **Prueba:** Evaluar el diseño (usabilidad, funcionalidad) y realizar ajustes.

Investigación de Usuarios

- **Encuestas:** obtienen datos cuantitativos y cualitativos directamente de los usuarios.
- **Información de ventas:** puede proporcionar ideas sobre qué productos o características son más populares entre los usuarios.
- **Información de mercadotecnia:** ayudan a comprender quiénes son los usuarios y cómo se relacionan con el producto.
- **Charlas de apoyo al usuario:** pueden revelar problemas recurrentes, dificultades y comentarios de los usuarios.

Perfil de usuario

- Edad, género, idioma, experiencia, ingresos, educación, localización, religión, ocupación, etc.

Diferencias entre UI y UX

UI	UX
Visual e interacción directa	Experiencia completa del usuario
Botones, íconos, colores	Usabilidad, navegación, empatía
Enfocado en estética	Enfocado en fluidez y funcionalidad

Ambos trabajan juntos: **UI es la parte visible, UX es la experiencia total.**

Ejemplo: el "Iceberg UX" muestra que la UI es solo la punta; lo más profundo es invisible pero fundamental.

Enfoque UI - Colores

Los colores son clave en la UI. Afectan la percepción, interacción y comodidad del usuario.

- Usar **pocos colores** (máx. 4-5 por ventana, 7 en total).
- Deben **apoyar la tarea** del usuario.
- Mantener **coherencia de gama**.
- Evitar **mezclas aleatorias**.
- Balance entre **saturación y brillo**.
- Considerar **discromatopsia** (~10% población).
- **Proveer información adicional** más allá del color.
- Usar **colores con significado coherente** en toda la UI.
- **Cambios de color** para mostrar estados (éxito, error).

Presentación de Información - Consideraciones

- **Conocer al usuario:** necesidades, objetivos, forma de interacción.
- **Información precisa vs. relaciones:** según contexto, usar números o comparaciones.
- **Inmediatez:** mostrar cambios en tiempo real cuando sea necesario.

- **Claridad para acciones:** si el usuario debe actuar, la info debe ser clara.
- **Texto vs. números:** elegir según facilidad de comprensión.
- **Información estática vs. dinámica:** adaptar la presentación según el comportamiento esperado.

Las 3 Reglas Doradas de Theo Mandel (1997)

1. Dar control al usuario

Permitir que los usuarios **tomen decisiones y personalicen su experiencia**. El sistema debe adaptarse al usuario, no imponerle restricciones. Esto implica:

- **Definir modos de interacción:** Reducir acciones innecesarias y facilitar el cumplimiento de tareas.
- **Interacción flexible:** Acomodar distintos estilos y preferencias del usuario.
- **Interrumpir y deshacer:** Ofrecer opciones para cancelar o revertir acciones, generando confianza.
- **Depuración progresiva:** Ajustar la interfaz según la experiencia y destreza del usuario.
- **Ocultar detalles técnicos:** No abrumar al usuario ocasional con información innecesaria.
- **Interacción directa:** Permitir interacción directa con los objetos en pantalla para mayor simplicidad y control.

2. Reducir la carga mental del usuario

El objetivo es diseñar **productos intuitivos y fáciles de usar**, minimizando el esfuerzo cognitivo. Estrategias clave:

- **Reducir demanda a corto plazo:** No sobrecargar con demasiada información o decisiones simultáneas.
- **Valores por defecto significativos:** Establecer predeterminados lógicos y funcionales.
- **Accesos directos intuitivos:** Crear atajos fáciles de aprender y usar.
- **Metáforas visuales:** Usar representaciones inspiradas en objetos del mundo real.
- **Desglose progresivo de información:** Presentar la información gradualmente conforme avanza la tarea.

3. Lograr una interfaz consistente

La **consistencia** mejora la familiaridad del usuario, reduce la curva de aprendizaje y minimiza errores. Para lograrla:

- **Contextualizar la tarea actual:** Ayudar al usuario a entender qué está haciendo y por qué.
- **Orientación y dirección:** Indicar claramente el punto de partida y los posibles caminos.
- **Consistencia entre aplicaciones:** Mantener uniformidad en todos los sistemas relacionados.
- **Reglas constantes:** Aplicar las mismas normas para situaciones similares.
- **Modelos prácticos:** Conservar modelos familiares salvo necesidad justificada.
- **Uniformidad visual:** Usar un mismo estilo para facilitar reconocimiento y uso.

Factores Humanos

Son esenciales para lograr productos que se adapten a las **capacidades físicas y cognitivas** de los usuarios.

Usabilidad

Es la **medida en que un sistema puede ser utilizado de forma efectiva, eficiente y satisfactoria**.

Según Donahue:

“La usabilidad es una medida de cuán bien un sistema de cómputo [...] facilita el aprendizaje, ayuda a recordar lo aprendido, reduce errores, permite eficiencia y deja satisfechos a sus usuarios.”

No depende de la estética ni de interfaces inteligentes por sí solas. Surge de cómo la **arquitectura de la interfaz se adapta a las personas**.

Principios de Usabilidad de Jacob Nielsen

Jacob Nielsen, referente en usabilidad, propuso una serie de principios fundamentales para establecer un **diálogo efectivo** entre el sistema y el usuario. Aunque surgieron para interfaces textuales, son completamente aplicables a interfaces gráficas y modernas.

1. Diálogo simple y natural

El sistema debe comunicarse con el usuario de forma **clara, coherente y natural**, sin redundancias ni complicaciones innecesarias.

Estrategias:

- Usar **escritura correcta**, evitando errores ortográficos o tipográficos.
- **Separar información relevante** de la irrelevante para no confundir al usuario.
- **Organizar** la información con una **distribución lógica**.
- Diseñar **prompts lógicos** y comprensibles que orienten al usuario.

- Evitar **mayúsculas excesivas y abreviaturas innecesarias** que dificulten la lectura.
- **Unificar funciones predefinidas** para mantener coherencia en acciones similares.

2. Hablar el lenguaje del usuario

La interfaz debe usar términos y expresiones familiares para el usuario, evitando tecnicismos.

Estrategias:

- Evitar **términos técnicos o extranjeros** innecesarios.
- No truncar palabras de forma excesiva.
- Diseñar entradas de datos con **etiquetas claras y comprensibles**.
- Proporcionar la **cantidad justa de información**, sin sobrecargar ni dejar dudas.

3. Minimizar la carga de memoria del usuario

Reducir el esfuerzo mental del usuario al interactuar, evitando que deba recordar información de una pantalla a otra.

Estrategias:

- Incluir **información de contexto**, ubicación en el sistema y estado de la sesión.
- Mostrar **rangos admisibles** en campos de entrada (por ejemplo, formatos válidos).
- Proporcionar **ejemplos o valores por defecto** como referencia.

4. Consistencia

La interfaz debe mantener una **coherencia visual, funcional y terminológica** para evitar confusión.

Estrategias:

- Mantener **consistencia visual** en colores, tipografías y estilos.
- Asegurar **consistencia tecnológica** en el comportamiento del sistema.
- Utilizar **terminología uniforme** en todas las partes del sistema.

5. Feedback (Retroalimentación)

El sistema debe **informar al usuario en todo momento** sobre lo que está ocurriendo.

Estrategias:

- Mostrar **estados de procesos** y progresos claramente.
- Comunicar el **estado del sistema y del usuario** (ej., sesión activa, operación exitosa).
- Incluir mensajes de **validación, confirmación o error** donde corresponda.
- **Validar los datos** ingresados e informar errores en tiempo real.

6. Salidas evidentes

El usuario debe poder **salir fácilmente de cualquier pantalla, tarea o estado**, sin quedar atrapado.

Estrategias:

- Incluir **botones visibles** para salir o cancelar en cada pantalla o contexto.
- Brindar **opciones claras** como deshacer, suspender, modificar.
- Asegurar que **siempre haya un camino de salida**, en cualquier momento del flujo.

7. Mensajes de error claros

Los errores deben ser comunicados **de forma útil, amigable y sin culpar al usuario**.

Estrategias:

- Mensajes de error **acorde al contexto**, explicativos y con posibles soluciones.
- **Evitar intimidaciones o tecnicismos**.
- **Categorizar errores** según su gravedad.
- Ubicar los mensajes en el **lugar y momento adecuados** sin interrumpir el flujo.

8. Prevención de errores

Es mejor **evitar que ocurran errores** a tener que corregirlos luego.

Estrategias:

- Usar **listas de selección** en lugar de campos de texto libres.
- Mostrar **ejemplos y formatos válidos**.
- Aplicar **validaciones automáticas**.
- Permitir cierta **flexibilidad de entrada**, como tolerar mayúsculas o símbolos comunes.
- Implementar **mecanismos de corrección automática** cuando sea posible.

9. Atajos

Los usuarios avanzados deben tener **alternativas más rápidas** para realizar tareas.

Estrategias:

- Incluir **atajos de teclado**, macros y **teclas de función**.
- Permitir **personalizar la interfaz** (menús, herramientas, accesos).

- Proporcionar **múltiples caminos** para lograr una misma acción.

10. Ayudas

El sistema debe ofrecer **ayuda accesible, útil y no intrusiva**, sin sobrecargar al usuario.

Estrategias:

- Asegurar la **disponibilidad** de ayudas contextuales o globales.
- Usar **diversidad de formatos**: texto, imágenes, videos, audio, FAQ, chat en línea.
- Permitir al usuario elegir el **tipo de ayuda** según su preferencia o necesidad.

Estilos de Interfaz

Cada **estilo de interfaz** representa una forma distinta en que el usuario puede **interactuar con un sistema**. La elección depende del tipo de aplicación, usuario objetivo y objetivos del diseño.

Interfaz de comandos

El usuario **ingresa texto** para interactuar con el sistema.

- **Ventaja**: Gran flexibilidad y poder.
- **Desventajas**: Difícil de aprender, **gestión de errores pobre**.
- Ej.: línea de comandos, terminal, intérpretes SQL.

Interfaz de selección de menú

El usuario **elige opciones predefinidas**.

- **Ventaja**: Reduce errores.

- **Desventaja:** Puede ser **lenta** para usuarios expertos.

Interfaz gráfica (GUI)

Usa **ventanas, botones, íconos y menús** con manipulación directa.

- **Ventajas:** Intuitiva, familiar, multitarea.
- **Desventajas:** Alto **consumo de recursos**, posible sobrecarga visual.

Interfaz de relleno de formularios

El usuario **completa campos específicos** con información estructurada.

- **Ventaja:** Claridad y facilidad de uso.
- **Desventaja:** Puede **ocupar mucho espacio** en pantalla.

Interfaz de manipulación directa

Interacción táctil o física con **elementos visuales**. Usada en dispositivos con pantallas sensibles.

- Ej.: cajeros automáticos, electrodomésticos con pantallas táctiles.

Interfaz de reconocimiento de voz

El usuario **habla y el sistema interpreta** comandos.

- Ej.: Siri, Google Assistant, Alexa.

Interfaz inteligente

Incorpora **inteligencia artificial** para adaptarse al usuario y mejorar con el tiempo.

- Capaz de **interpretar el contexto**, aprender del usuario y ofrecer respuestas personalizadas.

Interfaz responsive (adaptativa)

Se ajusta automáticamente a **diferentes tamaños de pantalla** y dispositivos.

- Garantiza **accesibilidad y consistencia** en computadoras, tablets, celulares, etc.

Interfaz accesible

Diseñada para ser utilizada por **usuarios con discapacidades físicas, sensoriales o cognitivas**.

- Promueve la **inclusión y la igualdad digital**.

PRUEBAS DEL SOFTWARE

Es un proceso sistemático y controlado que tiene como objetivo evaluar un programa de software, una aplicación o un sistema para identificar defectos, errores o fallos, y garantizar que cumpla con los requisitos y las expectativas establecidas

Finalidad de las pruebas

Una prueba se considera **exitosa** cuando logra descubrir un error. El propósito de esta fase es identificar errores que puedan comprometer la calidad del sistema, entendiendo que **ningún sistema está completamente libre de fallos**.

Las pruebas ayudan a identificar defectos que podrían surgir por cambios tecnológicos, errores de acceso, problemas de diseño, codificación u otros factores imprevistos. El objetivo es anticiparse a estos problemas para mejorar la calidad final del producto.

Objetivos

Los objetivos de las pruebas incluyen:

- **Detectar errores eficientemente.**
- Diseñar casos de prueba que permitan exponer distintas clases de errores.

- Reducir el tiempo necesario para encontrar defectos importantes.

Beneficios

Los beneficios clave de realizar pruebas incluyen:

- **Detección temprana de errores** antes del paso a producción.
- Reducción de **costos de mantenimiento**, ya que corregir errores antes de la entrega final es mucho menos costoso.
- Prevención de errores ocultos que podrían afectar al usuario final tras el lanzamiento.

Principios fundamentales de las pruebas de software

Las pruebas se rigen por varios principios que aseguran calidad y confiabilidad. Entre ellos se destacan:

- **Rastreabilidad:** Toda prueba debe estar vinculada a un requerimiento. Esto garantiza la trazabilidad entre requisitos, diseño y pruebas.
- **Planificación anticipada:** Es recomendable planificar las pruebas desde las etapas iniciales del desarrollo (por ejemplo, con historias de usuario).
- **Principio de Pareto:** El 80% de los errores proviene del 20% del código. Es útil identificar las áreas críticas y enfocarse en ellas.
- **Enfoque bottom-up:** Se sugiere comenzar probando **los módulos más pequeños**, y luego integrarlos en estructuras más complejas.
- **Cobertura exhaustiva:** Se deben cubrir todas las condiciones posibles a nivel de componentes para garantizar calidad.
- **Pruebas independientes:** Cuando sea posible, las pruebas deben estar a cargo de un **equipo independiente** al de desarrollo, para asegurar objetividad.

Equipo independiente de pruebas

Un equipo de pruebas independiente se compone de profesionales que verifican la calidad del software de manera **externa e imparcial** respecto al equipo de desarrollo. Entre sus beneficios:

- **Evita conflictos de interés:** Detectan errores sin estar condicionados por haber desarrollado el código.
- **Pruebas en paralelo a la codificación:** Esto **acelera** la detección y corrección de defectos.
- **Mejor colaboración:** La retroalimentación entre equipos asegura comprensión y solución eficaz de los problemas.

Errores: origen y tipos

Los errores pueden originarse por múltiples causas:

- Requerimientos ambiguos o incompatibles.
- Defectos de diseño (del sistema o del programa).
- Errores en el código implementado (algorítmicos, sintácticos, semánticos).
- Fallos en documentación, sobrecarga, capacidad, sincronización, recuperación, rendimiento, etc.

Tipos de defectos

- **Por omisión:** Cuando falta algún aspecto clave del código. (ej.: variable no inicializada).
- **Por comisión:** Ocurren cuando algún aspecto es incorrecto (ej.: valor asignado erróneo).

Tipos de pruebas de software

A grandes rasgos:

Pruebas de Caja Blanca (Glass Box Testing)

Este tipo de prueba se centra en la lógica interna del programa. El evaluador **tiene acceso al código fuente** y diseña casos de prueba que ejerciten caminos lógicos, decisiones, bucles y estructuras internas.

Objetivos:

- Ejercitar todas las decisiones lógicas del código.
- Ejecutar bucles y estructuras en sus **límites y casos extremos**.
- Evaluar estructuras internas de datos y control.

Se examinan rutas posibles, errores tipográficos, flujos no intuitivos o condiciones límite que podrían provocar errores difíciles de detectar a simple vista.

Prueba de Caja negra

La prueba de Caja Negra (también conocida como prueba cerrada) se enfoca en los requisitos funcionales del software y se lleva a cabo sin tener en cuenta el funcionamiento interno del programa. En las pruebas de Caja Negra, se examina la interfaz del software. Esto implica observar las entradas y salidas de datos, evaluando cómo responde el software a diferentes tipos de entradas. El tester no necesita conocer la lógica interna del código; solo se centra en la funcionalidad visible desde fuera. Concentra la prueba en el dominio de información y no en el desarrollo del algoritmo.

Estrategias de Prueba en el Desarrollo de Software

El **enfoque estratégico de pruebas** es fundamental para garantizar la calidad y la adecuación del producto final. Este enfoque proporciona una **guía detallada** para planificar, ejecutar y evaluar las pruebas a lo largo de todo el ciclo de vida del software.

Las estrategias de prueba están integradas a lo largo del proceso de desarrollo, desde el nivel más bajo (componentes individuales) hasta el nivel más alto (funcionamiento completo del sistema), asegurando la correcta verificación y validación del software.

Las pruebas no solo detectan errores, sino que también son parte del proceso de **aseguramiento de la calidad** del producto.

Conceptos clave

Verificación: Proceso que comprueba que el software ha sido construido correctamente en base a sus especificaciones. Involucra verificar que el diseño, el código y demás artefactos técnicos cumplan con lo planeado.

Validación: Proceso destinado a confirmar que el software satisface las **necesidades reales del cliente**, más allá de lo especificado técnicamente. Asegura que el producto sea útil y adecuado para su propósito final.

Pruebas de Unidad

Las **pruebas de unidad** evalúan el comportamiento individual de cada componente del sistema, verificando su lógica, datos internos y manejo de entradas y salidas. Se diseñan **casos de prueba específicos** que incluyen datos de entrada y valores esperados como salida.

Estas pruebas cubren:

- Verificación de la interfaz del módulo.
- Comprobación de estructuras de datos locales.
- Evaluación de condiciones límite.
- Ejecución de todos los **camino independientes** posibles del código.

Para ejecutar una prueba de unidad, se requieren dos elementos auxiliares:

- **Controlador:** Simula un programa principal, invoca al módulo, pasa los datos de entrada y recoge la salida.

- **Resguardo (stub):** Suple módulos subordinados. Emula el comportamiento de los módulos que aún no están implementados o que no son el foco de la prueba.

El uso de estos componentes permite aislar el módulo bajo prueba y evaluar su funcionamiento de forma controlada.

Un diseño modular con **alta cohesión** simplifica notablemente esta tarea, minimizando la necesidad de controladores y resguardos complejos.

En programación orientada a objetos, se prueba la **clase completa**, evaluando tanto su estado como las operaciones encapsuladas.

Pruebas de Integración

Las pruebas de integración se centran en evaluar la **interacción entre múltiples módulos** previamente probados de forma individual.

Posibles problemas detectados:

- **Pérdida de datos** en interfaces.
- **Efectos colaterales** entre módulos.
- **Errores en la secuencia de ejecución** de las funciones.

Para abordar estas pruebas:

- Se crean **módulos conductores** para invocar conjuntos de módulos.
- Se utilizan resguardos para simular componentes aún no desarrollados.
- Se realiza una **integración incremental**, agregando y evaluando módulos uno a uno.

Pruebas de regresión

Cada vez que se modifica el sistema (ya sea al agregar un módulo o al reemplazar un resguardo), se ejecutan **pruebas de regresión**. Estas aseguran que los cambios no

introduzcan nuevos errores en funcionalidades que previamente funcionaban correctamente.

Categorías de pruebas de regresión:

- **Componente:** Revisión directa del módulo modificado.
- **Contexto:** Abarca módulos relacionados que podrían verse afectados.
- **Software:** Evalúa una muestra representativa de funciones globales.

Módulos críticos a considerar

Deben priorizarse en las pruebas aquellos módulos que:

- Cubren múltiples requisitos.
- Controlan varios módulos subordinados.
- Tienen alta complejidad o historial de errores.
- Poseen requisitos de rendimiento específicos.

Estrategias de integración

Las pruebas de integración pueden seguir dos enfoques principales: **descendente** y **ascendente**. Dentro de cada uno se puede aplicar una estrategia **en profundidad** o **en anchura**.

En profundidad (Primero-en-profundidad)

1. Se crea el conductor desde el módulo raíz.
2. Se crean resguardos para todos los subordinados.
3. Se prueba la integración completa con resguardos.
4. Se reemplaza el primer resguardo por su módulo real.

5. Se ejecuta una **prueba de regresión**.
6. Se repite el proceso hasta integrar todos los módulos reales.

En anchura (Primero-en-anchura)

1. El conductor invoca todos los módulos hijos inmediatos del módulo raíz.
2. Se crean resguardos para estos módulos.
3. Se ejecuta la integración con resguardos.
4. Se reemplazan, uno por uno, los resguardos por sus módulos reales.
5. Se realiza una prueba de regresión tras cada reemplazo.

Estrategias de Prueba | Pruebas de Validación y Aceptación

Las pruebas de validación aseguran que el software desarrollado satisfaga completamente los **requisitos funcionales y no funcionales** establecidos. Dentro de esta categoría se encuentran las **pruebas de aceptación**, que son ejecutadas por los usuarios o clientes para validar si el sistema es apto para su uso en condiciones reales.

Tipos de Pruebas de Aceptación

Prueba Alfa	Prueba Beta
Realizada por el cliente .	Ejecutada por usuarios finales del sistema.
Se realiza en el entorno de desarrollo .	Se lleva a cabo en entornos reales de producción o en los lugares de trabajo de los clientes.

Entorno **controlado**: El uso del software simula un contexto real, pero bajo supervisión.

Entorno **no controlado**: El software se utiliza libremente en condiciones reales sin intervención directa del desarrollador.

El **desarrollador está presente** como observador, registrando errores o problemas.

El **desarrollador no participa** activamente. Los usuarios reportan los errores encontrados.

Se permite ajustar funciones y realizar cambios según observaciones.

Se hacen correcciones y mejoras a partir de los comentarios hasta que se define la versión final.

No es la versión final del software ni la última etapa de prueba

Constituye la **última fase** de pruebas. Utiliza técnicas de **caja negra** para probar la interfaz.

Pruebas en Entornos Personalizados

A medida que el software se vuelve más complejo, surge la necesidad de adoptar enfoques de pruebas especializados para garantizar la calidad del software en diversos aspectos.

Prueba de Arquitectura Cliente-Servidor

Evalúa la interacción y el correcto funcionamiento entre clientes (interfaces de usuario) y servidores (encargados del procesamiento de datos y ejecución de funciones).

Tipos de pruebas aplicadas:

- **Pruebas de funcionalidad**: Verifican que la aplicación funcione correctamente en su conjunto, especialmente en la interacción cliente-servidor.
- **Pruebas del servidor**: Evalúan desempeño, tiempo de respuesta y procesamiento bajo diferentes cargas de trabajo y situaciones de alta demanda.

- **Pruebas de base de datos:** Se verifica la precisión, integridad y consistencia de los datos almacenados.
- **Pruebas de transacciones:** Validan el correcto procesamiento de cada operación, incluyendo entradas y salidas.
- **Pruebas de comunicación de red:** Verifica la transmisión de datos, control de errores y desempeño de la red.

Pruebas de Sistemas en Tiempo Real

Estas pruebas se aplican a sistemas que deben reaccionar ante eventos con **restricciones temporales estrictas**.

Pruebas de Interfaces Gráficas (GUI)

Se enfocan en comprobar que la interacción con la interfaz gráfica del usuario sea funcional, intuitiva y visualmente correcta.

Prueba de la Documentación y Funciones de Ayuda

Este tipo de prueba verifica que el software esté **acompañado por documentación útil, clara y precisa**. Dos fases.

Depuración (Debugging)

La **depuración** es el proceso de identificar, analizar y corregir errores que se manifiestan durante las pruebas y a lo largo del ciclo de vida del software.

El proceso de depuración puede tener dos resultados: Se identifica la causa del error, se corrige y se elimina. No se logra identificar la causa del error de inmediato. En este caso, el encargado de la depuración puede tener sospechas sobre la posible causa. A partir de esto se diseñan casos de prueba adicionales para confirmar las sospechas y, en caso de confirmación, se procederá a corregir el error. Este proceso puede ser iterativo.

GESTIÓN DE LA CONFIGURACIÓN DEL SOFTWARE

Es un conjunto de prácticas y procesos utilizados en ingeniería de software para gestionar y controlar los cambios realizados en el software durante su ciclo de vida. El objetivo principal de la GCS es garantizar que el software se desarrolle, mantenga y entregue de manera coherente, controlada y rastreable.

Utilidad

Identifica el cambio, controla este, garantiza que el cambio se implemente adecuadamente e informa del cambio a todos aquellos que puedan estar afectados.

Línea Base

Es un punto de referencia fundamental en el desarrollo del software. Esta línea base se establece cuando uno o más Elementos de Configuración de Software (ECS) han sido completados y aprobados. Una vez que estos elementos se convierten en una línea base, cualquier modificación o cambio posterior en el documento se somete a la gestión y control del GCS.

Etapas del Proceso de GCS

1. **Identificación de elementos en la GCS**

Se identifican y definen los componentes del software a gestionar durante el proyecto. Cada uno incluye:

- Nombre claro y sin ambigüedad
- Tipo de ECS (documento, código fuente, datos)
- Identificador del proyecto
- Información de versión y/o cambio

2. **Control de versiones**

Uso de herramientas y procedimientos para rastrear y gestionar modificaciones sobre los ECS. Permite conocer qué se cambió, quién lo hizo, cuándo y por qué. Puede incluir ramificaciones del software.

3. **Control de cambios**

Proceso formal para solicitar, evaluar, aprobar y realizar cambios. La **ACC (Autoridad de Control de Cambios)** analiza el impacto en hardware, rendimiento, calidad, fiabilidad y satisfacción del cliente antes de aprobarlos.

4. **Auditoría de configuración**

Revisiones periódicas para evaluar el cumplimiento de los procesos de GCS y asegurarse de que las prácticas establecidas se sigan adecuadamente:

- Cambios coinciden con la orden de cambio.
- Se realizó una Revisión Técnica Formal (RTF).
- Se respetaron los estándares de Ingeniería de Software.
- Cambios registrados en ECS con fecha, autor y otros atributos relevantes.
- Procedimientos de GCS seguidos adecuadamente.
- ECS actualizados coherentemente.

5. **Generación de informes de estado de la configuración**

Consiste en mantener la documentación técnica y de usuario actualizada. Los informes responden a qué, quién, cuándo y cómo se realizaron los cambios y su impacto en el software.

MANTENIMIENTO

El mantenimiento es la **última etapa del ciclo de vida** del software, también llamada **Evolución del Sistema**. Incluye acciones como:

- Corregir errores detectados.
- Adaptar el software a necesidades del entorno.
- Incorporar mejoras mediante la modificación de partes del software.
- Optimizar su rendimiento.

Se aplica también a **sistemas heredados**, los cuales pueden ser antiguos, poco documentados y difíciles de modificar. Pueden ser desafiantes de mantener debido a su falta de claridad y organización.

Barrera de Mantenimiento

Llega un punto en el que continuar manteniendo un sistema deja de ser rentable. Esta **barrera** se evalúa comparando el costo del mantenimiento con el de desarrollar uno nuevo.

Desventajas del mantenimiento

- Puede limitar otros desarrollos.
- Cambios pueden generar nuevos errores o reducir la calidad.
- Requiere retomar fases como análisis o diseño.
- Puede representar entre el 40% y el 70% del costo total de desarrollo de un proyecto.
- Los errores en el software pueden provocar la insatisfacción del cliente.
- El trabajo de mantenimiento no siempre es atractivo, ya que no suele ser innovador ni creativo.
- En ocasiones, los cambios necesarios no fueron previstos en el diseño original.
- Es poco atractivo y difícil si el código no está documentado.

Tipos de mantenimiento

1. **Perfectivo (60%)**: Mejoras, nuevas funcionalidades y optimización. Es el tipo de mantenimiento más comúnmente utilizado
2. **Adaptativo (18%)**: Adaptación a nuevos entornos o plataformas. Migración de datos y funcionalidades a nuevas versiones o entornos.
3. **Correctivo (17%)**: Diagnóstico y corrección de errores detectados.
4. **Preventivo (5%)**: Cambios anticipados para evitar problemas futuros.

Actividades frente a un cambio

- Identificar el tipo de mantenimiento.
- Solicitar y documentar el cambio.
- Analizar su impacto y elaborar una hoja de ruta.
- Implementar y evaluar su adaptabilidad.
- Considerar el **efecto onda**.
- Probar exhaustivamente y liberar el sistema modificado.
- Diseñar cambios que sean fáciles de verificar y mantener.
- Finalmente, el sistema con las modificaciones es liberado y puesto en uso.

Ciclo de mantenimiento

Para llevar a cabo los cambios de manera efectiva, es esencial utilizar un mecanismo que permita identificarlos, controlarlos, implementarlos e informarlos. El proceso de cambio se simplifica cuando en el desarrollo inicial se han considerado atributos de calidad como modularidad y documentación interna del código fuente y de soporte.

1. **Análisis**: se define alcance, recursos, calidad y mejoras posibles.
2. **Diseño**: rediseño modular y claro, con notación estándar.

3. **Diseño detallado:** Se especifican estructuras de datos, interfaces, excepciones, y se consideran los efectos colaterales de la modificación.
4. **Implementación:** Se realiza la recodificación necesaria y se actualiza la documentación interna del código. Es fundamental seguir buenas prácticas.
5. **Pruebas:** Se realizan pruebas para revalidar el software después de los cambios.
6. **Actualización de documentación:** Revisar y actualizar cualquier documentación relacionada con el software que ha sido modificado.
7. **Distribución:** Una vez que se han realizado los cambios y se ha asegurado su calidad, las nuevas versiones del software deben ser instaladas y distribuidas.
8. **Notificación de cambio:** Es esencial comunicar de manera efectiva los cambios realizados en el software a los usuarios, clientes y otros involucrados.

Rejuvenecimiento del software

Acciones para **mejorar un sistema existente**, su objetivo es prevenir la degradación del sistema y evitar fallos relacionados con el envejecimiento, permitiendo aumentar su vida útil y continuar con el mantenimiento del software.

- **Re-documentación:** Analizar el código existente para generar documentación que revele su estructura, flujo y funcionamiento.
- **Re-estructuración:** Reorganiza el software para hacerlo más comprensible y manejable.
- **Ingeniería inversa:** Reconstruye documentos de diseño desde el código. Se utiliza cuando no existe documentación para comprender el sistema.
- **Re-ingeniería:** Extensión de ingeniería inversa, comienza con código desconocido y a partir de la misma genera nuevo código mejorado sin alterar su funcionalidad.

Es un **análisis crítico** para evaluar eficiencia y eficacia, identificar debilidades y recomendar posibles cursos de acción que mejoren la organización y ayuden a alcanzar los objetivos establecidos.

Tipos:

- **Interna:** la realiza alguien dentro de la organización.
- **Externa:** aporta mayor objetividad y capacidad de identificar aspectos que quizás no fueron detectados internamente.

En muchos casos, se recomienda utilizar ambos enfoques.

Funciones clave:

- Proteger activos (hardware, software, personal).
- Asegurar integridad y seguridad de los datos.
- Garantizar eficiencia y cumplimiento de objetivos.
- Prevenir delitos, pérdidas y vulnerabilidades.

Campo de acción:

- Incluye la evaluación **administrativa del área**, de los **sistemas y procedimientos**, y de la **eficiencia en el uso de la información**. También abarca el **proceso de datos**, los **equipos y sistemas informáticos** (software, hardware, redes, bases de datos, comunicaciones), así como aspectos de **seguridad, confidencialidad y legales** relacionados con los sistemas y la información.